Outil d'analyse de langages

Hategekimana Fabrice

09 mars 2021

Motivation:

Ce que nous voulons:

- Étude de langage
- Sémantique opérationnelle

Créer un outil pour:

- Définir la sémantique d'un langage
- En voir ses dérivations

Outil

langage:

- Syntaxe simplifiée pour la définition de langage
- Utilisation de PLY en tant que lexer/parser

interface

- CLI -> GUI
- Outils de debbuging qui prévient l'utilisateur
- Visualisation par graphe

Actuellement

developpement

```
- [ ] redéfinition + clarification de la grammaire
- [ ] gérer les erreurs (recursion depth, confusion variables/
- [ ] module d'aide à la correction des erreurs
```

rédaction

```
- [ ] documentation (structure du projet)
```

langage

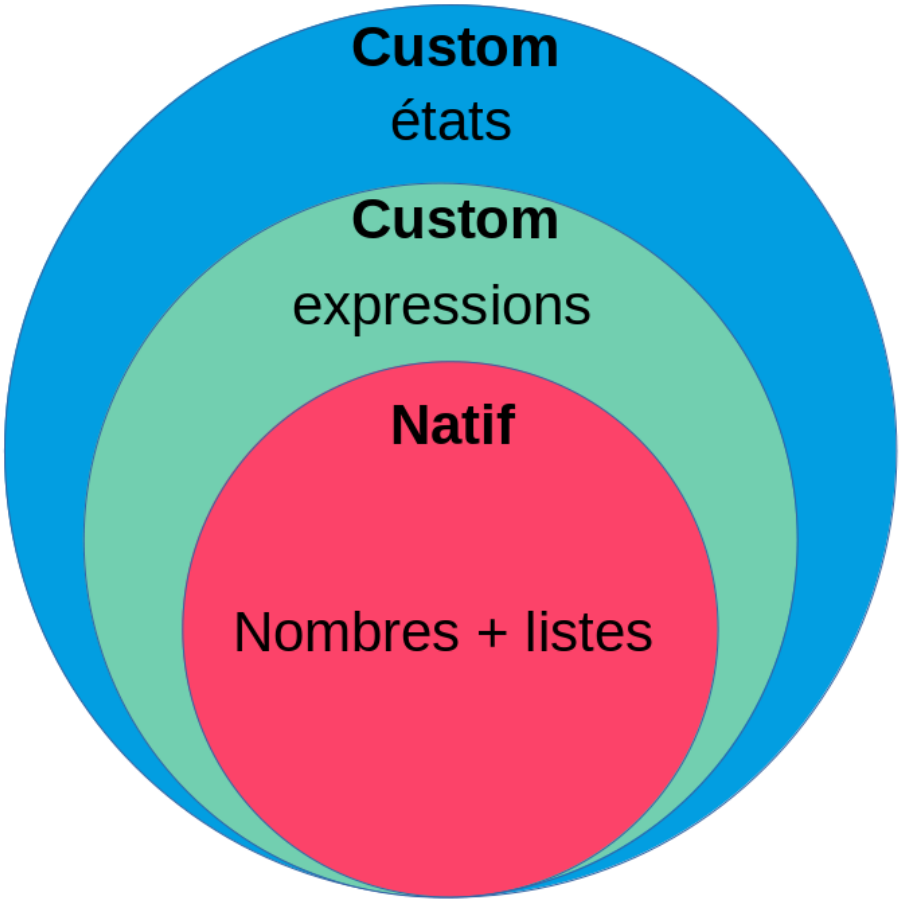
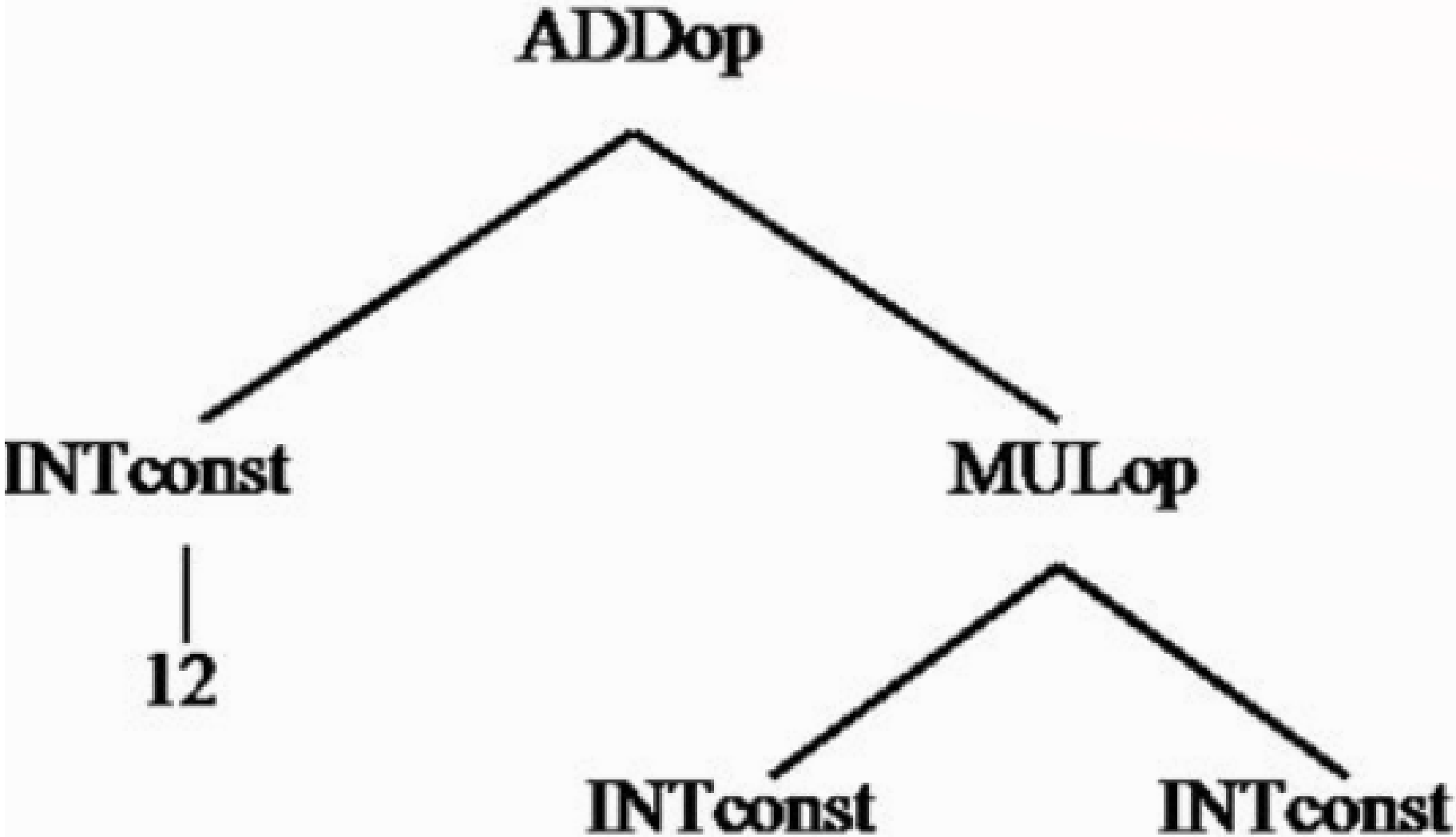


Figure 1: langage_ensemble_shema

langage natif (1)

Tree-based abstract syntaxe



langage natif (2)

Arithmétique simple

- addition **add(a,b)**
- soustraction **sub(a,b)**
- division **div(a,b)**
- multiplication **mul(a,b)**

langage natif (2)

gestion de liste

liste: [], [1,2,3], ...

exemple

l1= [1,2,3], **l2= [4,5,6]**

- accès **get(l1, 0)** => 1
- définition **set(l2, 1, 7)** => [4,7,6]
- insertion **insert(l1, 0)** => [0,1,2,3]
- ajout **append(l2, 7)** => [4,5,6,7]
- retrait **remove(l2, 1)** => [4,6]
- retrait **removeLast(l2)** => [4,5]
- retrait **pop(l2)** => 6
- retrait **pop(l2, 0)** => 4

Pour le reste

$$\textsc{Cas} + : \quad \frac{modify(Tab, Pt, 1) = Tab'}{\langle Tab, Pt, + \rangle \Rightarrow_I \langle Tab', Pt \rangle}$$

Traduction Idéale

```
modify(Tab,Pt,1) = TabP -- <Tab,Pt,+> => <TabP,Pt>
```

Réalité

```
-- <Tab,Pt>plus => <modify(Tab,Pt,1),Pt>
```

détails

Nombres:

```
- Le langage supporte actuellement les nombres relatifs
```

Booléens:

```
- Le langage supporte les booléens (True et False)
```

Liste:

```
- Ne peut contenir que des nombres/variables/booléens
```

Variable:

```
- Doit obligatoirement commencer par une majuscule
```

fonction:

```
- Doit obligatoirement commencer par une minuscule
```

architecture

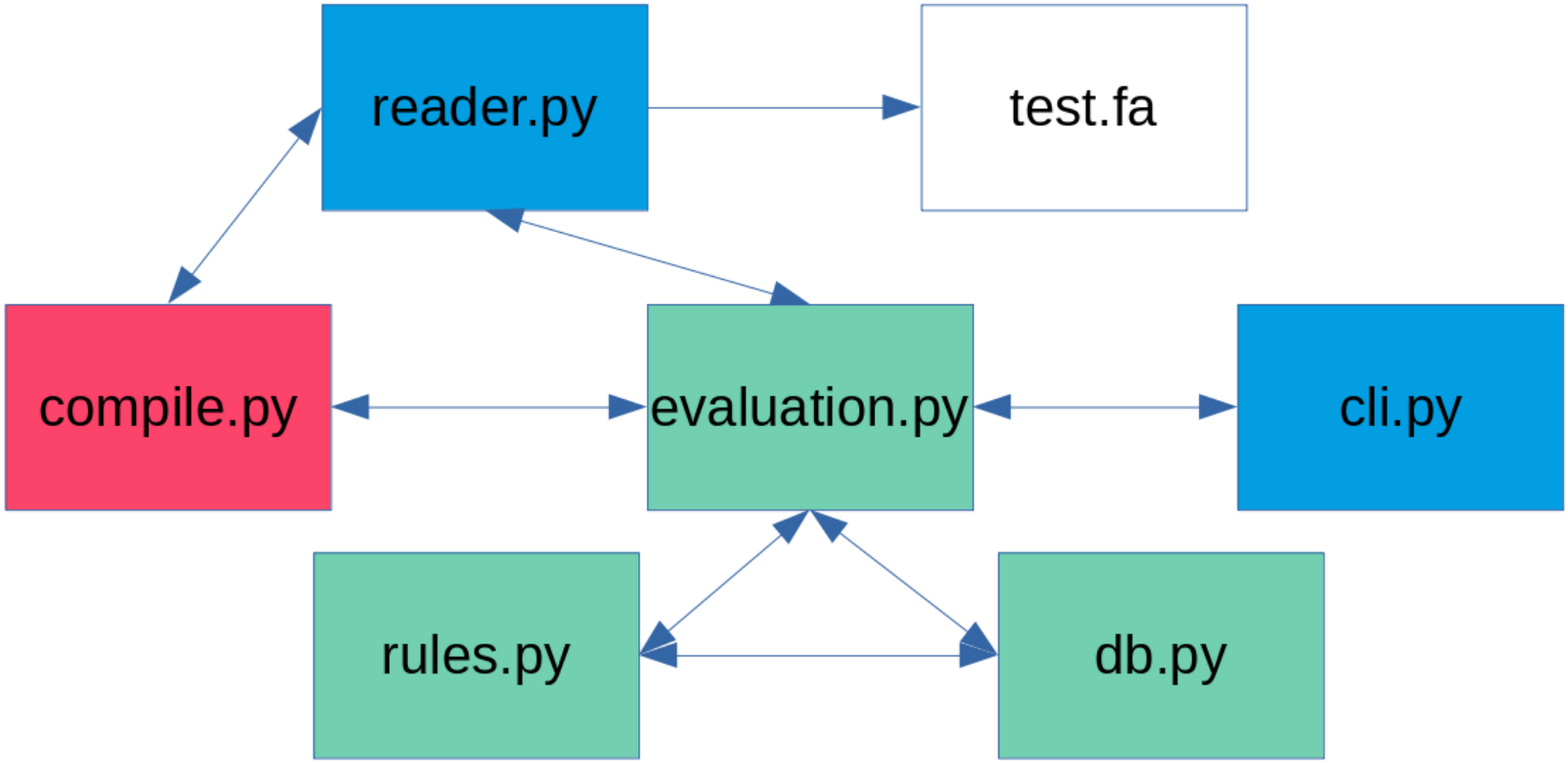


Figure 3: architecture

factorielle.fa

règles

$$\frac{}{fact(1) = 1}$$

$$\frac{N > 1}{fact(N) = mul(N, fact(sub(N, 1)))}$$

traduction

```
-- fact(1) = 1
N > 1 -- fact(N) = mul(N,fact(sub(N,1)))
```

len.fa

règles

$$\frac{}{len([]) = 0}$$

$$\frac{L \rightarrow Lp, Lp \in list}{Len(L) = add(1, len(removeLast(Lp)))}$$

traduction

```
-- len([]) = 0
L->Lp, Lp in list -- len(L) = add(1,len(removeLast(Lp)))
```

max.fa

règles

$$\frac{A \rightarrow Ap, B \rightarrow Bp, Bp >= Ap}{max(A, B) = Bp}$$

$$\frac{A \rightarrow Ap, B \rightarrow Bp, Ap >= Bp}{max(A, B) = Ap}$$

$$\frac{}{maxL([]) = 0}$$

$$\frac{}{maxL(L) = max(pop(L), maxL(removeLast(L)))}$$

traduction

```
A -> Ap, B -> Bp, Bp >= Ap -- max(A,B) = Bp
A -> Ap, B -> Bp, Ap >= Bp -- max(A,B) = Ap
-- maxL([]) = 0
-- maxL(L) = max(pop(L), maxL(removeLast(L)))
```

Syntaxe State (du domaine sémantique)

State

```
<e1,...,en>
```

State x inst

```
<e1,...,en>inst
```

Transition

```
<e1,...,en>inst -> <i1,...,in>
```

Le compteur

règles

$$\frac{Compteur \in number}{< Compteur > plus = < add(Compteur, 1) >}$$

$$\frac{Compteur \in number}{< Compteur > plus = < sub(Compteur, 1) >}$$

traduction

```
Compteur in number -- <Compteur>plus = <add(Compteur,1)>
Compteur in number -- <Compteur>moins = <sub(Compteur,1)>
```

Le point

règles

$$\frac{}{< X, Y > gauche = < sub(X, 1), Y >}$$

$$\frac{}{< X, Y > droite = < add(X, 1), Y >}$$

$$\frac{}{< X, Y > bas = < X, sub(Y, 1) >}$$

$$\frac{}{< X, Y > haut = < X, add(Y, 1) >}$$

traduction

```
-- <X,Y>gauche = <sub(X,1),Y>
-- <X,Y>droite = <add(X,1),Y>
-- <X,Y>bas = <X,sub(Y,1)>
-- <X,Y>haut = <X,add(Y,1)>
```

mode debug

Principe:

Un peu similaire au trace de prolog. Le but est de permettre à l'utilisateur de voir pas à pas le developpement de son programme pour trouver les failles.

Permet de:

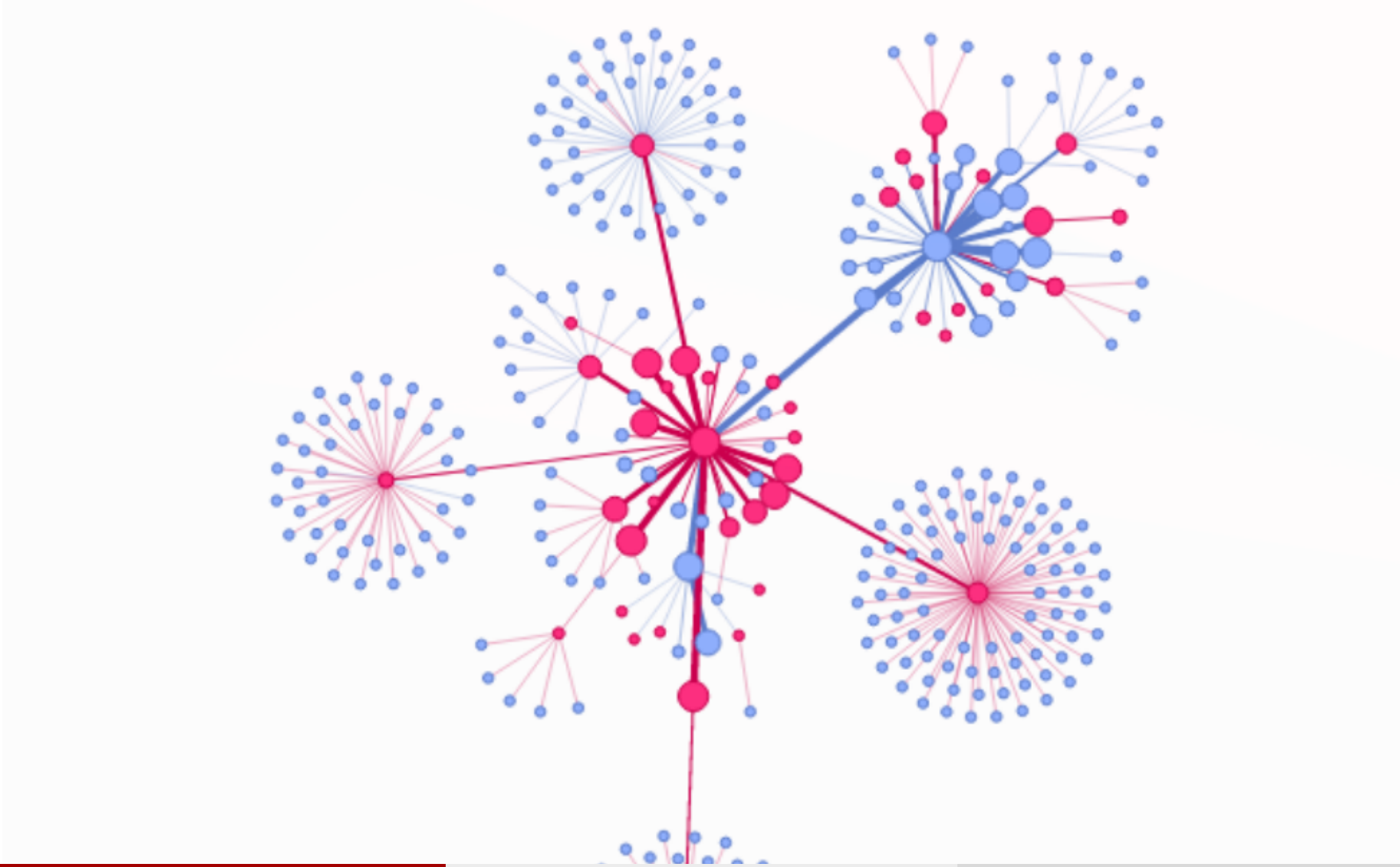
- voir l'état de l'exécution
- voir les règles applicables
- visualiser le parcours d'exécution
- (faire des test)

Instructions

- import [path]: importe un fichier [nom].fa
- set exp [exp]: donne une expression à évaluer
- set state state : donne un état à évaluer
- step: va à l'étape suivante
- end: poursuit l'exécution jusqu'au bout
- state: voir l'état courant du système d'évaluation

Interface (visualisation par graphe)

pyvis



outils de visualisation

Affichage

`NetworkX + pyvis + vis.js`

Problèmes

algorithme d'évaluation

```
- récursif vs boucle
- granularité de l'évaluation
```

Dans les premisses

```
Mauvaise évaluation des expressions
```

Document

Structures:

1. principes de base
2. motivation
3. principes et concepts théorique
4. présentation de l'outils
5. documentation de l'outils
6. conclusions

Pour la suite

21.04.21

- interface du système de dérivation (suite)
- système de gestion des erreurs
- écriture de la documentation (suite)

28.04.21

- système de gestion des erreurs (suite)
- faire tester l'outil
- écriture de la documentation (suite)

05.05.21

- écriture de la documentation (suite)
- faire tester l'outil
- système de gestion des erreurs (suite)

12.05.21